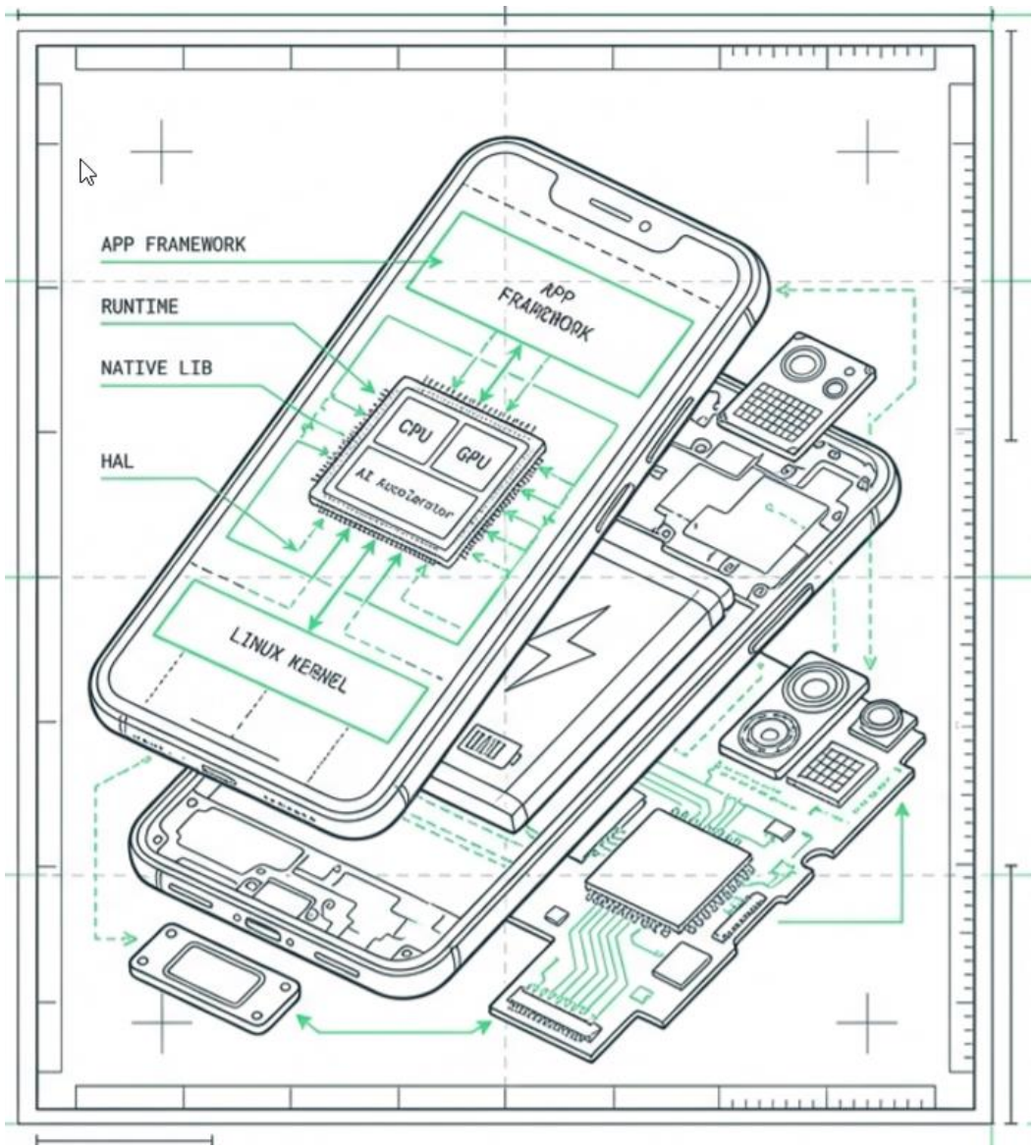
Android 系統開發基礎簡介

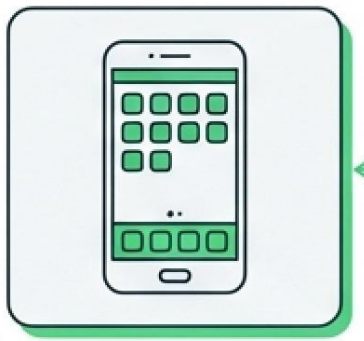
系統架構與開發入門

探索全球最廣泛使用的行動作業系統。
從架構原理到實作流程的完整藍圖



Fred Chiang
DAMAA0
2026.02.26

為什麼選擇 Android？



全球影響力

全球市占率最高
的行動作業系統(70% ~ 75%)，
開發技能具備極高價值。

多元應用場景

跨足平板、電視、車載
系統與物聯網(IoT)。

系統深度

學習 Android 不僅能
打造 App，更能理解
作業系統背後的邏作
邏輯。

系統架構概覽

The 5-Layer Architecture

Application(應用程式層)

Framework(系統框架層)

Android Runtime
(ART / Dalvik)

HAL(硬體抽象層)

Linux Kernel(核心層)

Android 系統由這五層堆疊而成，每一層都向下依賴並向上提供服務

深入解析系統分層

Linux Kernel	提供記憶體管理、程序排程、驅動程式支援(The Engine)
HAL (硬體抽象層)	將硬體功能封裝成統一介面，讓上層軟體無須關心硬體實作細節 (The Interface)
Android Runtime	負責執行應用程式的 Java/Kotlin 程式碼(The Executor)
Framework	提供開發者 API，如 Activity 管理、通知系統(The Toolkit)
Application	使用者實際操作的 App 所在層級(The User Experience)

開發環境與必備工具

Android Studio

官方整合開發環境(IDE)，
支援 Kotlin 與 Java

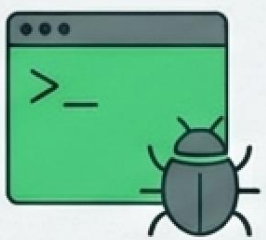
Gradle

專案建置工具，負責管理套件
依賴與編譯流程。

模擬器與真機

(Emulator/Device)

快速部署測試與實機驗證。

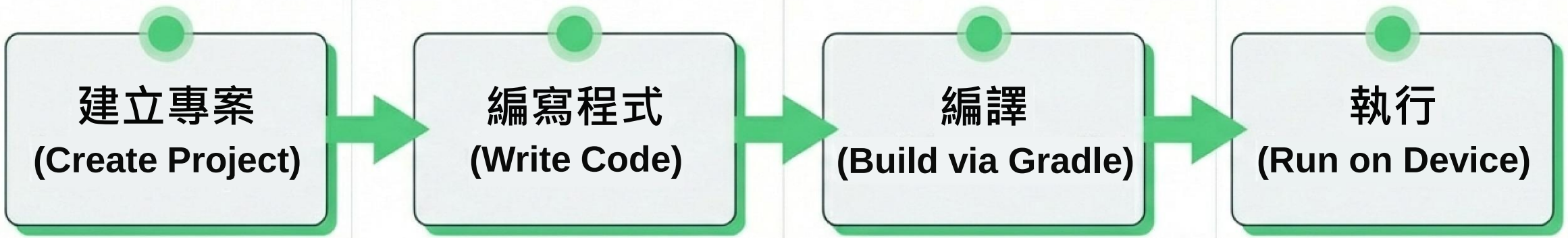


除錯工具

(Logcat / Profiler)

Logcat 輸出日誌，Profiler
分析記憶體與 CPU 效能。

基礎開發流程：從 Hello World 開始



Main Activity(入口)

App 的主要程式入口。

```
public class MainActivity extends AppCompatActivity {
    7 usages
    private static final String TAG = "AndroidCameraApi";
    3 usages
    private Button takePictureButton;
    7 usages
    private TextureView textureView;
```

資源與編譯

- 資源管理(Resources)：圖片與設定檔集中管理（res/layout）
- 編譯執行：透過 Gradle 打包並安裝APK

UI 設計與資源管理

activity_main.xml

```
<Linear Layout ..>
    <Button
        android:text='Click Me'
        android:layout_width="wrap_content' />
    <Text View
        android:text="'Hello World' />
</Linearl Layout>
```

資源抽離(Resource Isolation)：

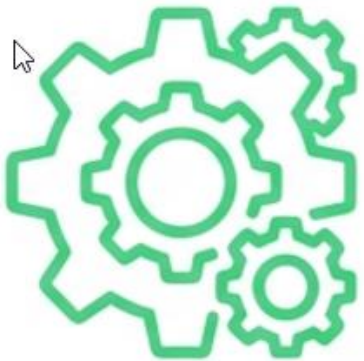
- strings.xml(文字字串)
- draw able/(圖片資源)
- styles.xml(視覺風格)

Android 四大核心元件

Core Components

Activity

負責處理畫面顯示
與使用者互動
(UI&Interaction)

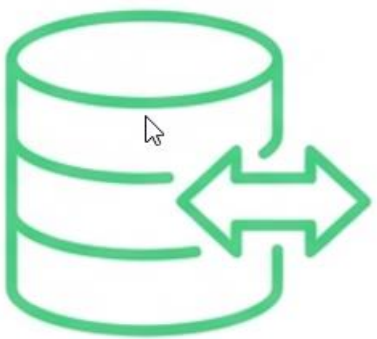


Service

負責在背景執行長時間任務，
例如音樂播放
(Background Tasks)

Broadcast Receiver

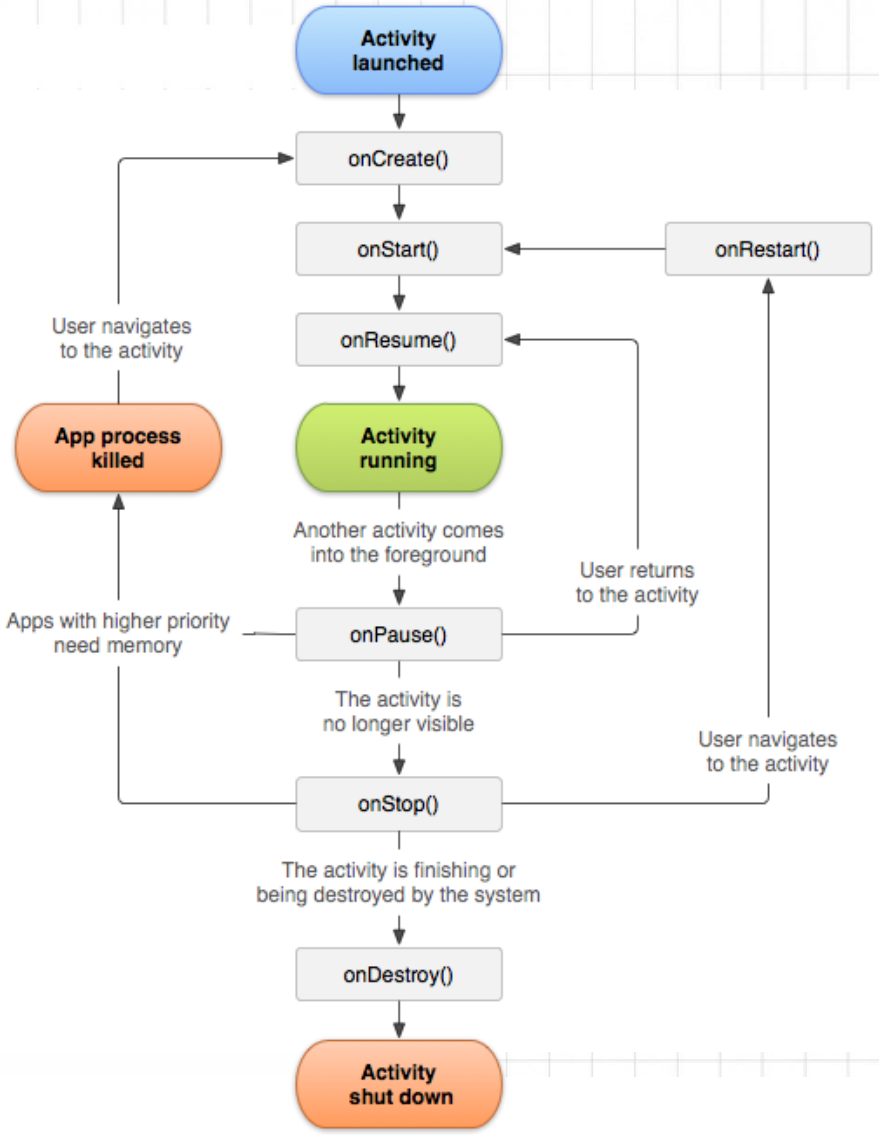
接收系統或 App 發出的廣播
訊息，如電量低警告
(System Events)。



Content Provider

負責跨 App 之間的資料
共享與存取
(Data Sharing)

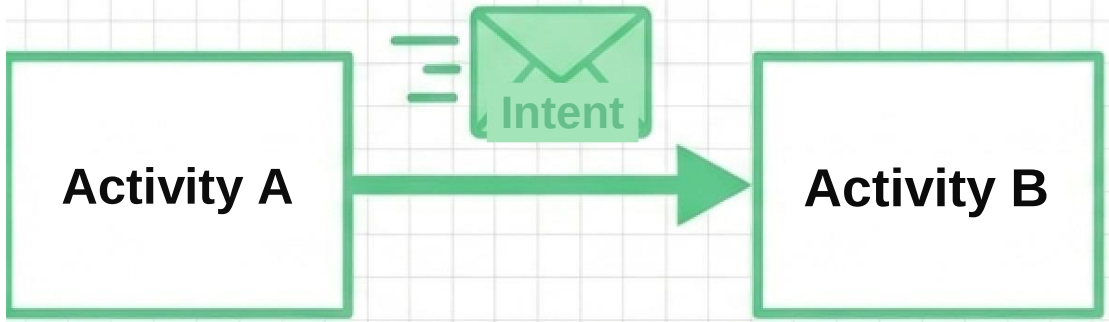
Activity 生命週期



理解生命週期是防止
App 崩潰與優化效能的關鍵

元件通訊與權限宣告

Intent(通訊機制)



元件間傳遞訊息的機制，可用於切換畫面或傳送資料

Android Manifest.xml(身分與權限)

App 的身分證。宣告元件與系統權限

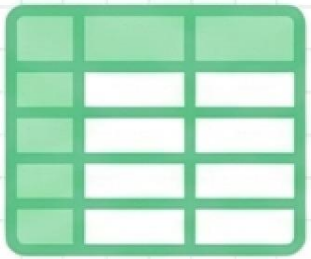
宣告權限(Permissions)：如相機、網路

Runtime Permission:危險權限需動態請求

資料儲存方式

Shared Preferences

適合儲存少量設定資料
(Key-Value)



SQLite / Room

適合儲存結構化的大量資料。
Room 是官方推薦的資料庫抽象層

File Storage

適合儲存圖片、文件等
大型檔案

重點觀念檢測

Q：Android 系統的最底層是什麼？

A: Linux Kernel

Q：哪一個元件負責背景執行長時間任務？

A: Service

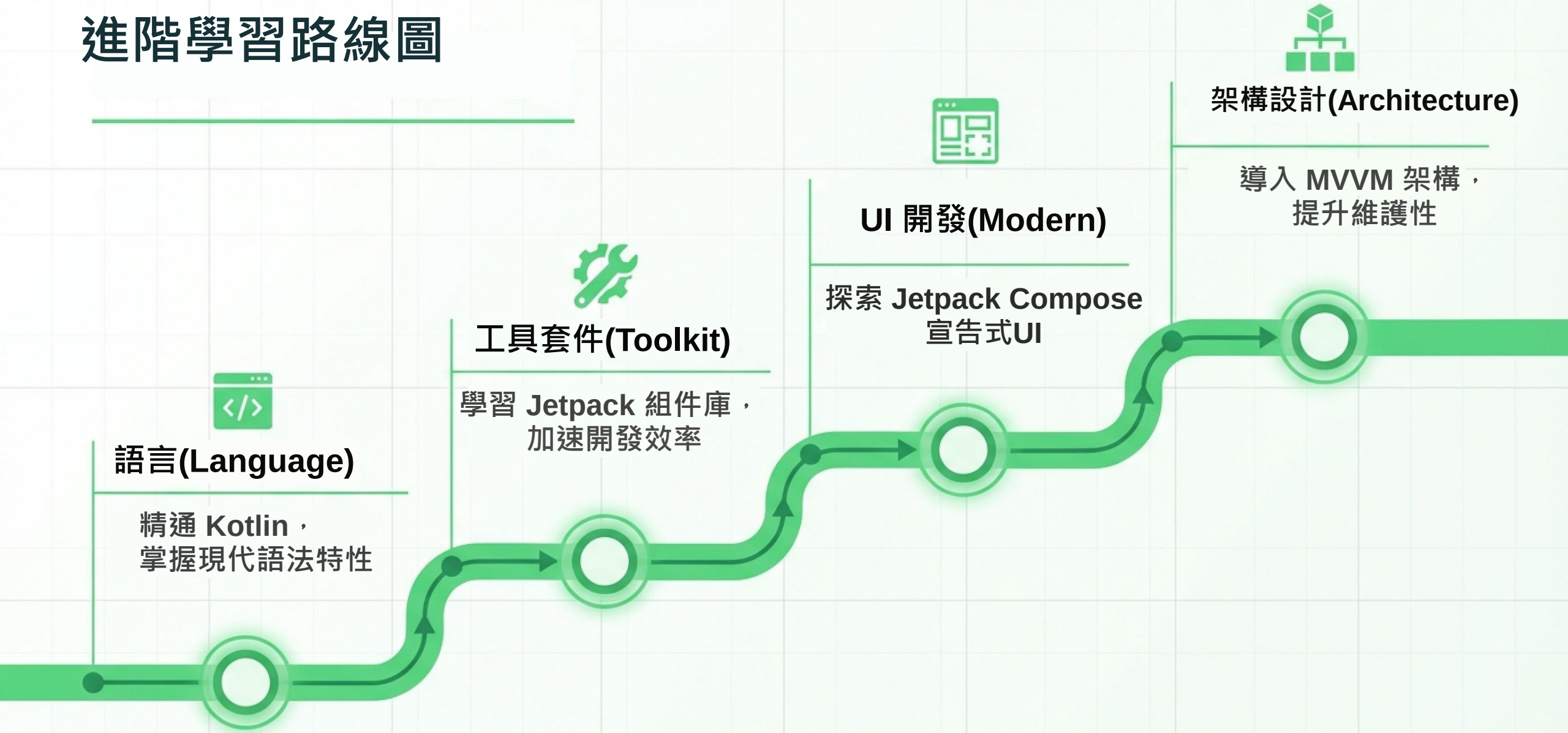
Q:官方最推薦使用哪種語言開發 Android？

A: Kotlin

Q：權限需要在哪個檔案中宣告？

A: Android Manifest.xml

進階學習路線圖


語言(Language)
精通 Kotlin，
掌握現代語法特性
工具套件(Toolkit)
學習 Jetpack 組件庫，
加速開發效率
UI 開發(Modern)
探索 Jetpack Compose
宣告式UI
架構設計(Architecture)
導入 MVVM 架構，
提升維護性

總結與展望

- 已掌握五層系統架構(System Architecture)
- 熟悉 Android Studio 與 Gradle 工具鏈(Tools)
- 理解 Activity 生命週期與四大核心元件(Core Components)

這是你打造下一個世界級 App的起點
持續練習，動手實作！